

Game Ops System for Multiplayer Game Event

Problem Understanding

The objective is to design a backend system capable of processing multiplayer match data during a large-scale game event.

The system provides:

- Fair and competitive leaderboards
- Suspicious player detection
- Balanced matchmaking
- Scalable architecture capable of supporting growth from thousands to hundreds of thousands of players

The solution focuses on backend engineering principles including scalability, maintainability, reliability, testability, and clean system design.

Assumptions

- Match results are submitted by trusted game servers.
 - The game supports Solo, Duo, and Squad modes.
 - Maximum players per match = 100.
 - Players belong to a region and device category.
 - Suspicious players are flagged for review rather than automatically banned.
 - Matchmaking balances fairness and queue time.
 - Leaderboards are maintained independently for Solo, Duo, and Squad modes.
 - Rankings are updated after every match submission.
-

Data Structure / Database Design

Database

PostgreSQL

Players

- player_id (PK)
- region
- device
- created_at

Player Mode Stats

Stores aggregated statistics and rankings separately for each game mode.

- id (PK)
- player_id (FK)
- game_mode
- ranking_score
- total_score
- total_kills
- total_deaths

Matches

- match_id (PK)
- game_mode (SOLO / DUO / SQUAD)
- match_duration_seconds
- created_at

Match Results

- id (PK)
- player_id
- match_id
- ping
- score
- kills
- deaths

Suspicious Players

- player_id
- anomaly_score
- avg_kill_efficiency
- avg_score_efficiency
- avg_kd_ratio
- created_at

Matchmaking Queue

- player_id
- region
- device_class
- game_mode
- ping
- skill_rating
- queue_entered_at

Indexes

Indexes are maintained on:

- player_id
 - region
 - ranking_score
 - skill_rating
 - game_mode
-

Leaderboard Logic

The leaderboard follows an esports-inspired ranking model.

Ranking Formula

Ranking Score =

$(\text{Kills} \times 1000) + \text{Score} - (\text{Deaths} \times 50)$

Tie Breakers

When ranking scores are equal:

1. Higher Kills
2. Higher Score
3. Fewer Deaths

Leaderboard Types

The system supports:

- Global Leaderboard

- Region Leaderboard
- Mode Leaderboard
- Region + Mode Leaderboard

Examples:

- Global
- India
- SEA
- Europe
- SOLO
- DUO
- SQUAD
- India + SOLO
- SEA + DUO
- Europe + SQUAD

Performance Optimization

To avoid expensive leaderboard recalculations:

- Ranking scores are stored in PlayerModeStats.
- Scores are incrementally updated after each match.
- PostgreSQL indexes ranking_score for efficient sorting.
- Leaderboards support pagination.

Example:

GET /leaderboard?page=1&limit=100

Suspicious Player Detection Logic

The system uses population-relative anomaly detection instead of fixed thresholds.

For every player:

Kill Efficiency

Kills / Match Duration

Score Efficiency

Score / Match Duration

KD Ratio

$\text{Kills} / \max(\text{Deaths}, 1)$

These metrics are compared against players within the same:

- Region
- Device category

An anomaly score is generated based on deviation from the population average.

A player is not flagged from a single exceptional match.

The system evaluates multiple matches before generating a suspicious player record, reducing false positives and helping distinguish highly skilled players from abnormal behavior.

Matchmaking Logic

The matchmaking system uses an Expanding Matchmaking Matrix.

Step 1: Pool Isolation

Players are grouped using:

(region, device_class, game_mode)

Examples:

- (India, Mobile, SOLO)
- (Europe, PC, SQUAD)

This prevents unfair cross-device, cross-region, and cross-mode matchmaking.

Step 2: Continuous Skill Rating (SR)

SR =

$\text{Average Score} + (\text{Average Kills} \times 10) - (\text{Average Deaths} \times 5)$

Player Rank Tiers

- Bronze
- Silver
- Gold

- Platinum

These tiers are displayed to players.

Actual matchmaking uses the underlying SR value.

Step 3: Dynamic Queue Relaxation

Wait Time	Ping Range	SR Range
0–5 sec	±20 ms	±100
6–15 sec	±50 ms	±300
>15 sec	±100 ms	Any SR

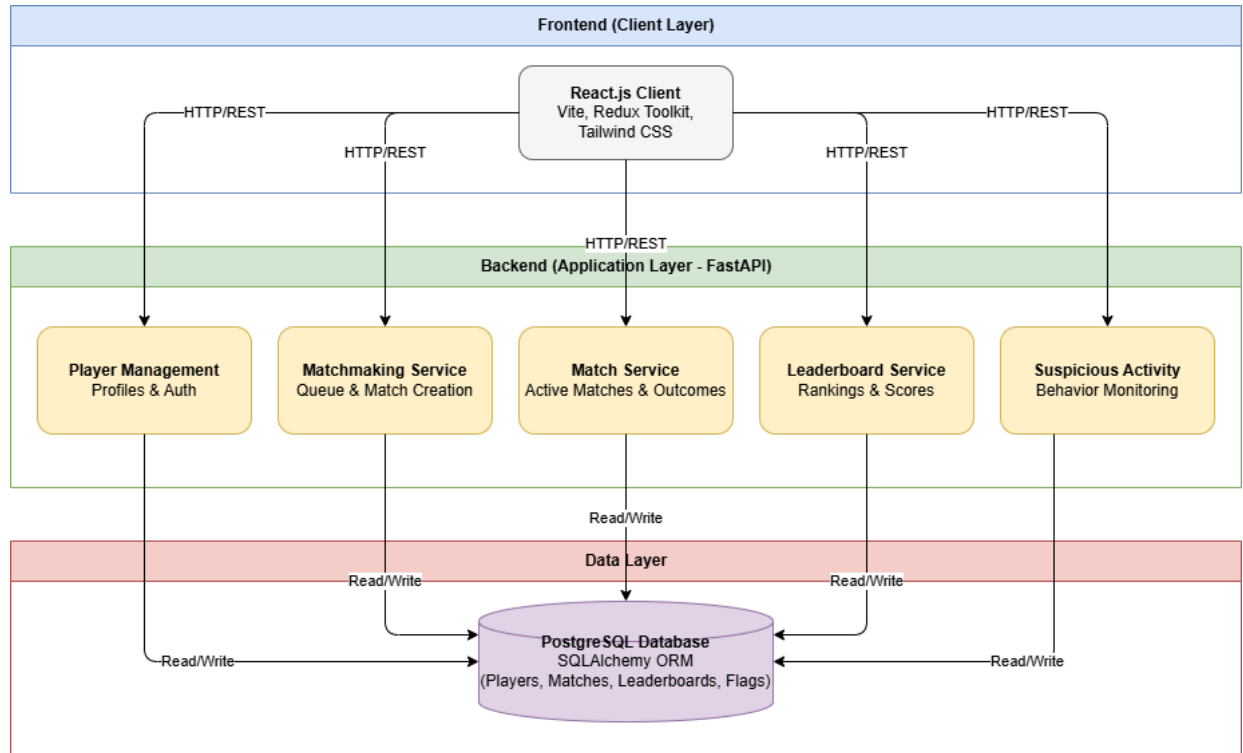
This prevents matchmaking deadlocks while prioritizing fair matches.

Queue Cleanup

When a lobby is successfully formed:

- Selected players are removed from the active queue.
 - Queue removal and lobby creation occur within the same database transaction.
 - Duplicate matchmaking is prevented using transactional locking.
-

Architecture Overview



Scaling Plan

Current Scale (~2,000 Players)

- Single FastAPI instance
- PostgreSQL database

Medium Scale (20,000–50,000 Players)

- Database indexing
- Query optimization
- Pagination
- Precomputed leaderboard updates
- Efficient `ranking_score` indexing

Large Scale (200,000+ Players)

Load Balancer

↓

FastAPI Instances



PostgreSQL

Multiple FastAPI instances can be added horizontally without modifying business logic because APIs remain stateless.

Monitoring

Metrics tracked:

- API latency
- Request throughput
- Error rates
- Database performance
- CPU utilization
- Memory utilization

Tools:

- Prometheus
 - Grafana
-

Limitations

- Sophisticated coordinated cheating may bypass anomaly detection.
- Continuous Skill Rating is simpler than advanced systems such as ELO or TrueSkill.
- Region isolation may increase queue times in low-population regions.
- PostgreSQL may eventually require partitioning or read replicas at very large scale.
- Leaderboards are near real-time rather than fully real-time.
- Suspicious player detection remains a review mechanism and is not an automatic anti-cheat system.

Despite these limitations, the system provides a practical, scalable, maintainable, and production-oriented backend architecture suitable for large multiplayer game events.

Validation & Quality Metrics

The implementation was validated using automated unit and integration tests.

Test Execution Summary

Metric	Value
Total Tests	129
Passed	129
Failed	0
Overall Coverage	93%
Backend Framework	FastAPI
Testing Framework	Pytest
Coverage Tool	pytest-cov

Test Coverage by Module

Module	Coverage
Config & Core Utilities	97-100%
Models	100%

Routers	94-100%
Matchmaking Service	99%
Suspicious Detection Service	98%
Player Service	90%
Leaderboard Service	78%
Match Service	77%
Matchmaking Utilities	96%
Statistics Utilities	100%

Functional Areas Tested

Leaderboard

- Ranking score calculation
- Tie-breaker ordering
- Global leaderboard
- Region leaderboard
- Mode leaderboard
- Region + Mode leaderboard
- Pagination

Match Processing

- Match creation
- Match validation

- Maximum 100-player validation
- Ranking score updates

Suspicious Player Detection

- Kill efficiency calculation
- Score efficiency calculation
- KD ratio calculation
- Population-relative anomaly detection
- Multi-match evaluation
- Suspicious player persistence

Matchmaking

- Queue join
- Queue leave
- Duplicate prevention
- Dynamic relaxation logic
- Lobby creation
- Queue cleanup
- Transaction safety

Player Management

- Player creation
- Player retrieval
- Player listing
- Validation handling

Quality Assurance

The system follows:

- Repository Pattern
- Service Layer Pattern
- Dependency Injection
- SQLAlchemy 2.0
- Pydantic Validation
- Transaction Ownership in Services
- Loguru-based Logging

Automated testing and 93% code coverage provide confidence in the correctness, reliability, and maintainability of the implementation.